

参赛编号： 006026

论文题目：A 题 光伏电站发电功率日前预测问题

光伏电站发电功率日前预测问题的研究

摘 要

光伏电站发电功率日前预测是提升电网调度效率和运行安全的关键技术。本研究围绕气象条件、地理分布及季节变化对预测精度的影响，分别通过**太阳辐照理论、季节指数预测模型、卷积神经网络和长短期神经网络的混合模型**，系统构建了多模型融合的预测框架，并验证了空间降尺度技术的有效性。

针对问题 1，本文从光伏电站的地理位置信息与设备参数出发，基于太阳的高度角，方位角辐射计算理论，构建了光伏电站理论发电功率计算模型。并将理论功率与实际功率对比定义偏差函数。在发电功率特性不同周期跨度研究上。本文对长周期采用**基于局部加权回归的季节趋势分解 (STL)**，对短周期采用小波分析进行时间序列分解，发现了发电功率受季节因素影响显著，年度功率趋势成分呈现“**双峰**”形态，而日内发电功率则与昼夜更替紧密相关，呈现典型的“**钟形**”曲线趋势的特性。最后结合实际频率和理论频率的偏差模型，得出了受气象因素关键影响等光伏电站发电功率特性。

针对问题 2，本文基于问题 1 中发电功率周期变动的特性构建了**季节指数预测模型**。问题 2 是基于历史发电功率的预测问题，首先根据原始时间序列中数据的变动趋势拟和趋势方程反应数据的长期趋势，然后对相关数据进行**趋势比率**方式消除趋势，接着对同“季节”数据进行**简单平均和正规化处理**消除随机干扰，获得季节指数数据。于是，可根据趋势方程和季节指数构建预测模型。最后通过均方根误差、平均绝对误差和平均误差等指标量化模型的准确性。

针对问题 3，本文融合了天气气象数据构建了发电功率**卷积神经网络和长短期神经网络的混合预测模型 (CNN-LSTM)**。首先对气象数据采用相关度分析，对其降维处理，以选取合适的指标与问题 2 中的时序向量进行后期训练模型。接着为了避免局部误差最优和误差震荡的现象出现，对数据进行**归一化处理**消除极大值和量纲的干扰。先通过卷积神经网络对输入的气象数据进行**空间特征的提取**，再使用长短期神经网络对输出的空间向量特征和历史功率的时序数据进行预测模型的训练。最后引入全天空指数深入剖析模型基于不同气象条件下的性能表现，根据**全天空指数**进行聚类处理，划分晴天、多云、阴雨等气象状况。针对每一类气象条件，分别提取样本进行预测后，使用问题 2 中指标来验证模型的准确性得到显著提升，。

针对问题 4，针对不同光伏电站的地理经纬度数据和实际气象数据，采用**克尔金插值**的方式依次完成变异函数的拟和和克里金方程的求解，从而得到**克尔金插值**空间模型。本文采取对光伏电站覆盖生成表格的方式，对每个网格点执行克尔金插值，获得降维后的气象数据，同时对每季节重新拟和变异函数，适应季节性气象变化。据此，将空间降尺度数据带入问题 3 的混合预测模型进行预测；并将数据按照不同场景划分，将空间降尺度数据和普通数据准确率结果进行对比，结果显示多云场景预测效果显著提升。

关键词： STL 时序分解 小波分析 季节指数预测 卷积神经网络 克尔金插值

一、 问题重述

1.1 问题背景

光伏发电通过半导体材料的光电效应将太阳能转化为电能，光伏电站作为规模化发电设施，其发电功率主要由光伏板接收的太阳辐射总量决定，呈现双重波动特征：受季节更替导致的太阳光倾角变化影响，辐照强度存在季节性长周期波动；而云量、阴雨、雾霾等气象因素则引发分钟至小时级的短周期波动。

当光伏电站接入电网时，其功率波动会直接影响电网的功率平衡与频率稳定，例如突发气象变化可能导致发电功率骤变，若调度滞后易引发供需失衡甚至系统风险。因此，精准预测未来 24-48 小时的发电功率（日前预测）是电力调度的核心需求，可帮助优化机组启停、储能调度等策略，保障电网安全并提升可再生能源消纳效率。

当前，光伏功率日前预测面临显著挑战：一方面，云系移动、局地降水等中小尺度气象过程的不确定性，导致数值天气预报（NWP）对辐照强度的预测误差较大，直接影响功率预测精度；另一方面，地理特征（如经纬度、海拔、光伏板倾角）与气象参数（辐照、温度、湿度）的非线性耦合作用复杂，传统建模方法难以有效刻画功率生成机制。此外，兆瓦级光伏电站的局部尺度与传统 NWP 千米级空间分辨率存在不匹配问题，进一步加剧了预测难度。

因此，如何基于历史功率数据与 NWP 信息，构建融合物理机制与数据驱动的预测模型，解析长短期波动规律，突破多源数据耦合与空间尺度失配的瓶颈，提升光伏功率日前预测的精度与鲁棒性，成为当前电力工程领域亟待解决的关键问题，对推动高比例可再生能源并网具有重要理论与实际意义。

1.2 问题提出

问题一：结合地理位置与太阳辐照理论，分析光伏电站发电功率的季节性长周期、日内短周期特性，通过实际与理论功率偏差揭示发电特性。

问题二：基于历史功率数据建立光伏电站日前预测模型，按要求划分训练/测试集，分析预测方法的准确性。

问题三：融入 NWP 信息构建预测模型，对比评估其是否提升预测精度；若提升，给出场景划分方案并验证。

问题四：采用机器学习等方法对 NWP 数据进行空间降尺度，检验其能否提高光伏功率预测精度，分析可行性及原因。

二、 问题分析

问题一：首先，需建立光伏电站理论发电功率计算模型。光伏电站发电功率与太阳辐照强度密切相关，而太阳辐照强度受地理位置、太阳运行轨迹等因素影响。因此，需结合电站地理位置信息，运用太阳高度角、方位角辐射计算理论，构建理论发电功率模型，为后续分析提供基准。

其次，需研究发电功率在不同周期的特性。在长周期（季节性变化）层面，季节更替导致气象条件改变，进而影响发电功率，可采用基于局部加权回归的季节趋势分解（STL）方法，剖析年度功率趋势成分；在短周期（日内波动）层面，发电功率受昼夜影响呈现周期性变化，利用小波分析对时间序列分解，捕捉日内功率波动特征。

最后，需分析实际功率与理论功率的偏差。通过定义偏差函数，对比实际功率与理论功率，结合实际频率和理论频率的偏差模型，探究气象等因素对发电功率特性的影响，明确关键影响因素，为电站运行优化与功率预测提供依据。

问题二：首先，以历史功率序列为核心输入构建预测模型，鉴于该序列具有强自相关性、日 / 周周期性及非平稳性，需先对数据进行异常值剔除（如 3σ 法则）和缺失值修复等预处理。其次，在模型选择上，传统时序模型（如 ARIMA）适用于线性趋势与季节周期刻画，但对非线性波动拟合能力有限；深度学习模型（如 LSTM）虽擅长捕捉复杂时序模式，却存在过拟合风险，因此可采用“分解 - 集成”框架融合多时间尺度特征。最后，依据题目要求划分训练集与测试集，将误差统计限定于白昼时段，基于考核指标（如 MAE、RMSE）验证模型在不同季节、天气场景下的预测精度，通过对比分析确保模型在峰值时段（如午间）和全天气场景中的泛化能力。

问题三：首先，通过相关性分析筛选与太阳高度角相关系数大于 0.7 的 NWP 气象变量（如辐照度、气温、风速），利用主成分分析降维后与历史功率时序特征融合，构建包含时空信息的复合输入矩阵。其次，采用 CNN-LSTM 等融合算法，通过 CNN 提取气象数据的空间特征（如辐照度分布相关性），借助 LSTM 捕捉功率序列的时间依赖特性，并通过注意力机制强化对高波动时段的特征学习。最后，对比纯时序模型与融入 NWP 的模型误差，若预测精度显著提升（如 RMSE 降低 $\geq 15\%$ ），则进一步按天气类型或季节划分场景验证，分析 NWP 信息在高不确定性场景（如多云天气）中的增益效果，明确其对预测精度的提升机制。

问题四：首先，针对传统 NWP 数据空间分辨率（千米级）大于光伏电站覆盖面积的问题，采用统计降尺度、机器学习或空间插值等方法获取更小尺度的气象信息，以匹配电站局部微气候需求。其次，通过对比原始 NWP 模型、降尺度模型与纯时序模型的预测误差，检验降尺度方法的可行性，若误差显著降低（如 RMSE 降低 $\geq 10\%$ ），则证明其有效。最后，结合电站地理特征（如地形、城市环境）分析原因，由于降尺度技术减少了大尺度数据“空间平均”导致的误差，能更精准反映电站实际辐照度与温度，尤其在复杂地形或突发气象场景中，可显著提升预测精度，从而验证空间降尺度对光伏功率预测的优化作用。

三、 模型假设与符号说明

3.1 模型基本假设

- (1) 数据集准确，可信度高。。
- (2) 电机未老化或损坏，运行符合标定参数
- (3) 大气散射效应可忽略，能够基于地理位置和太阳高度角推测理论可发功率。
- (4) 发电站附近无极端恶劣天气，光伏功率波动遵循固定的周期性模式。
- (5) 光伏电站覆盖区域内的气象条件（如辐射、温度）在百米尺度下具有空间平稳性，可通过克里金插值准确估计。

3.2 符号说明

符号	含义	符号	含义
P	光伏发电功率	S	季节指数
δ	太阳赤纬角	K	全天气指数

ω	时角	D	两点距离
t_{solar}	真太阳时	γ	半方差
h	太阳高度角	z	气象观测值
α	太阳方位角	N	点对数量
β	光伏面板倾角	λ	权重
θ	太阳光与面板夹角	P_{sc}	理论最大发电功率
G	太阳直接辐射强度	A	光伏组件总面积
η	能量转换效率	γ	方位角
Δ	功率偏差值	C_R	准确率
T	长期趋势值	Q_R	合格率

四、模型一的建立与求解

4.1 模型准备

4.1.1 数据集分析

本文进行研究的 dataset 来源于科学数据银行的光伏发电量 PVOD 数据集，该数据集共有 271,968 条记录，由两个间隔为 15 分钟的数据源构建，即气象服务的数值天气预报（NWP）和光伏电站的本地测量数据（LMD）。具体参数如下：

数据规格	参数值
数据来源（公开数据集链接）	https://www.scidb.cn/en/detail?dataSetId=f8f3d7af144f441795c5781497e56b62
光伏电站装机容量	6600 MW
发电功率和 NWP 数据时间分辨率	采样点/15min
发电功率和 NWP 数据起始-截止时间（一年）	2019.01.01- 2019.12.31
NWP 属性	气温、辐射、云量

表 1 数据集具体信息

4.1.2 数据预处理

在获取 PVOD 数据集的光伏电站功率数据后，需对数据进行统计分析与预处理，以保障数据质量。

对于缺失值处理，鉴于时间序列数据可能出现偶发缺测，本研究采用前向填充（ForwardFill）方法，即利用缺失值前的最近观测值进行填补，以此维持数据连续性。若出现连续多个数据缺失，前向填充无法反映真实数据特征时，则将对对应样本从数据集中剔除。

在无效与异常数据剔除环节，通过设定物理合理性约束条件进行筛选。发电功率必须为非负值；太阳辐射强度不超过太阳常数（ 1361W/m^2 ）；若温度、湿度等气象数据出现负值等异常情况，则对其进行删除或修正，确保输入模型的数据在物理意义上保持一致，为后续分析奠定可靠基础。

4.2 理论可发功率计算模型的建立

首先建立地球绕太阳运动中太阳赤纬角 δ 与时角 ω 的计算公式：

$$\delta(n) = 23.45 \cdot \sin\left(\frac{2\pi}{365} \cdot (284 + n)\right)$$
$$\omega(t_{\text{solar}}) = 15 \cdot (t_{\text{solar}} - 12)$$

其中：

· $n \in [1, 365]$ 表示一年中的某一天，

· t_{solar} 表示太阳时。

因此太阳高度角 h 和太阳方位角 α 的表达式可以表示为：

$$h(t) = \arcsin(\sin \phi \cdot \sin \delta + \cos \phi \cdot \cos \delta \cdot \cos \omega)$$
$$\alpha(t) = \arccos\left(\frac{\sin \delta \cdot \cos \phi - \cos \delta \cdot \sin \phi \cdot \cos \omega}{\cosh}\right)$$

之后可以计算太阳光与面板法线的夹角 θ 为：

$$\cos \theta(t) = \sinh(t) \cdot \cos \beta + \cosh(t) \cdot \sin \beta \cdot \cos(\alpha(t) - \gamma)$$

其中：

β 为板面的倾斜角，

γ 为方位角。

则根据太阳辐照理论可知，单位面积接受到的太阳直接辐射强度 S 由太阳光与法线的夹角 θ 决定（忽略大气散射时近似），计算公式为：

$$G(t) = G_0 \cdot \cos \theta(t)$$

其中：

G_0 为太阳常数，近似为 1361W/m^2 。

最后理论可发功率 P_y 可以表示为：

$$P_y(t) = \eta \cdot A \cdot G(t)$$

其中：

η 为光伏组件的能量转换效率， $\eta \in (0, 1)$ ， A 为光伏组件的总面积。

4.3 发电功率规律初步分析

通过绘制日内发电功率平均曲线及季节总发电功率图，初步揭示光伏电站发电功率的周期性特征：

日内短周期：日出至正午（6:00-12:00）功率呈快速增长趋势并达到峰值；正午至日落功率缓慢下降至零；夜间（18:00 - 次日 6:00）因无太阳辐照，功率为 0，呈现典型 “钟形” 曲线。

季节长周期：春季（3-5 月）功率呈上升趋势，夏季（6-8 月）达到最高值，秋季（9-11 月）功率下降且为四季中最低，冬季（12-2 月）功率有所回升但仍处于较低水平。

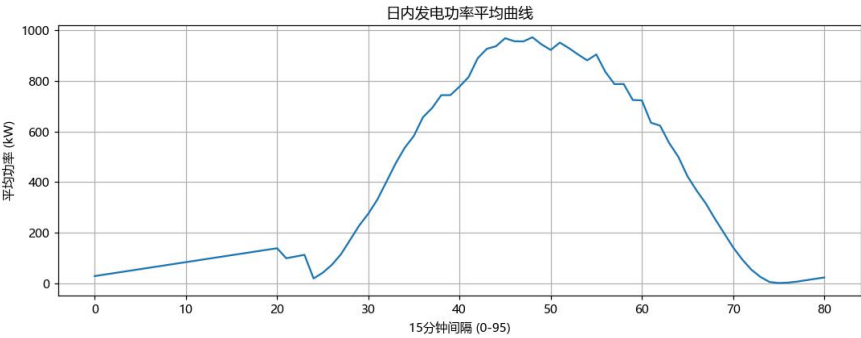


图 1 日内发电功率平均曲线

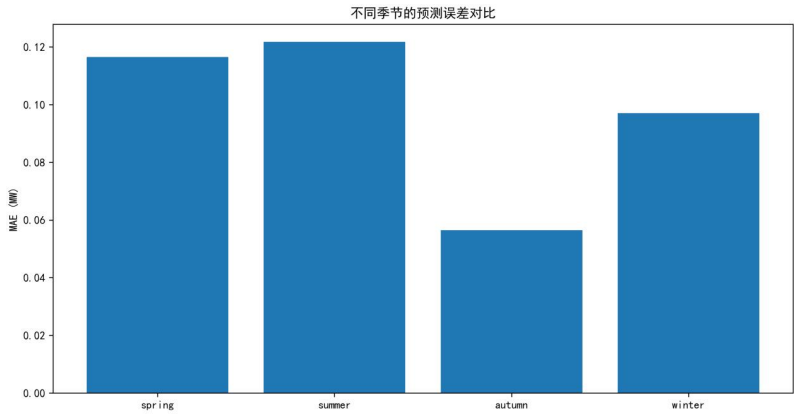


图 2 季节总发电功率图

4.4 长周期变化分析模型（STL 分解）的建立

STL 是一种以鲁棒局部加权回归 (Locally weighted scatterplot smoothing, LOESS) 作为平滑方法的时间序列分解方法，基于 LOESS 将时间序列数据分解为趋势项 (Trend)、季节项 (Seasonal) 和残差项 (Residual)。与其他季节分解方法（比如 X11[28] 分解法）相比，STL 方法对异常点处理鲁棒性更强，能处理加法模式的分解。本题中 STL 分解通过分离光伏发电偏差的长期趋势、日内季节性和随机波动，为分析季节性变化和异常事件提供了清晰视角。在光伏电站运维中，趋势项可用于制定年度发电计划，季节项优化日内调度策略，而残差项则帮助快速定位设备故障或突发气象影响。

STL 由内循环和外循环嵌套而成，内循环主要拟合趋势和计算周期，外循环主要用来调节鲁棒权重。其核心是内循环的 LOESS 迭代，主要过程如下图所示。

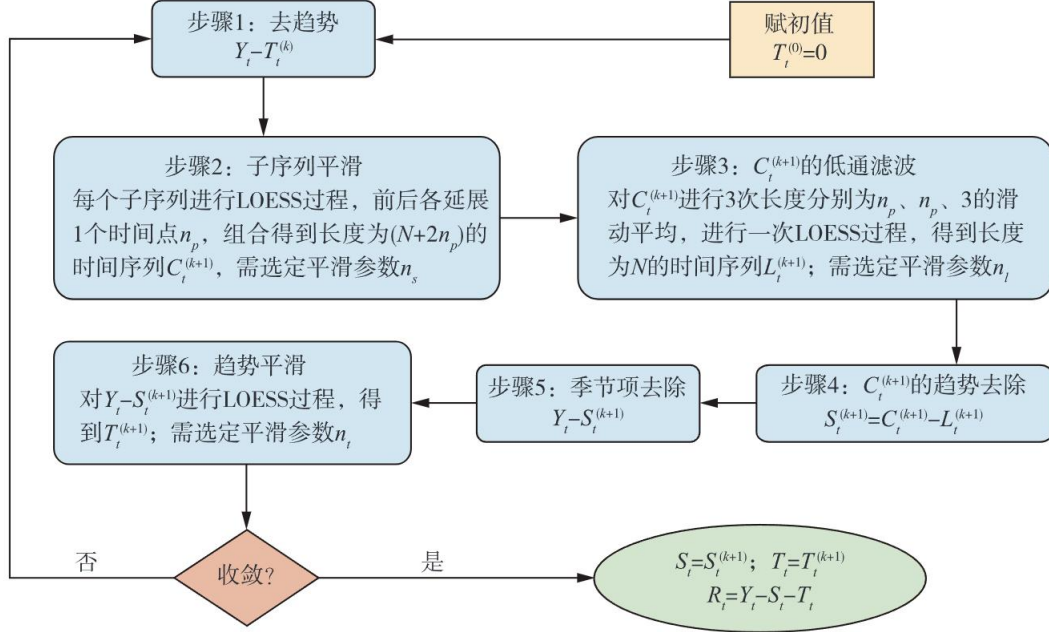


图 3 STL 循环图

对于本题，按照日数关系进行聚类分析，再按照每月进行 STL 分解。
首先对全年功率差按天聚合：

$$\Delta_d = \sum_{i=1}^{96} \Delta_{\text{real}}(d, i)$$

然后再对按月进行平均：

$$\Delta_m = \frac{1}{N_m} \sum_{d \in \text{month } m} \Delta_d$$

最后进行 STL 分解的方式找到相关特性：

$$\Delta(t) = T(t) + S(t) + R(t)$$

T 为趋势项，其反映数据的长期变化方向。

S 为季节项，固定周期的重复波动。

R 为残差项，去除趋势和季节影响后的随机波动。

然后对三者分别作出对时间的拟和曲线，反应长周期特性，得到结果。

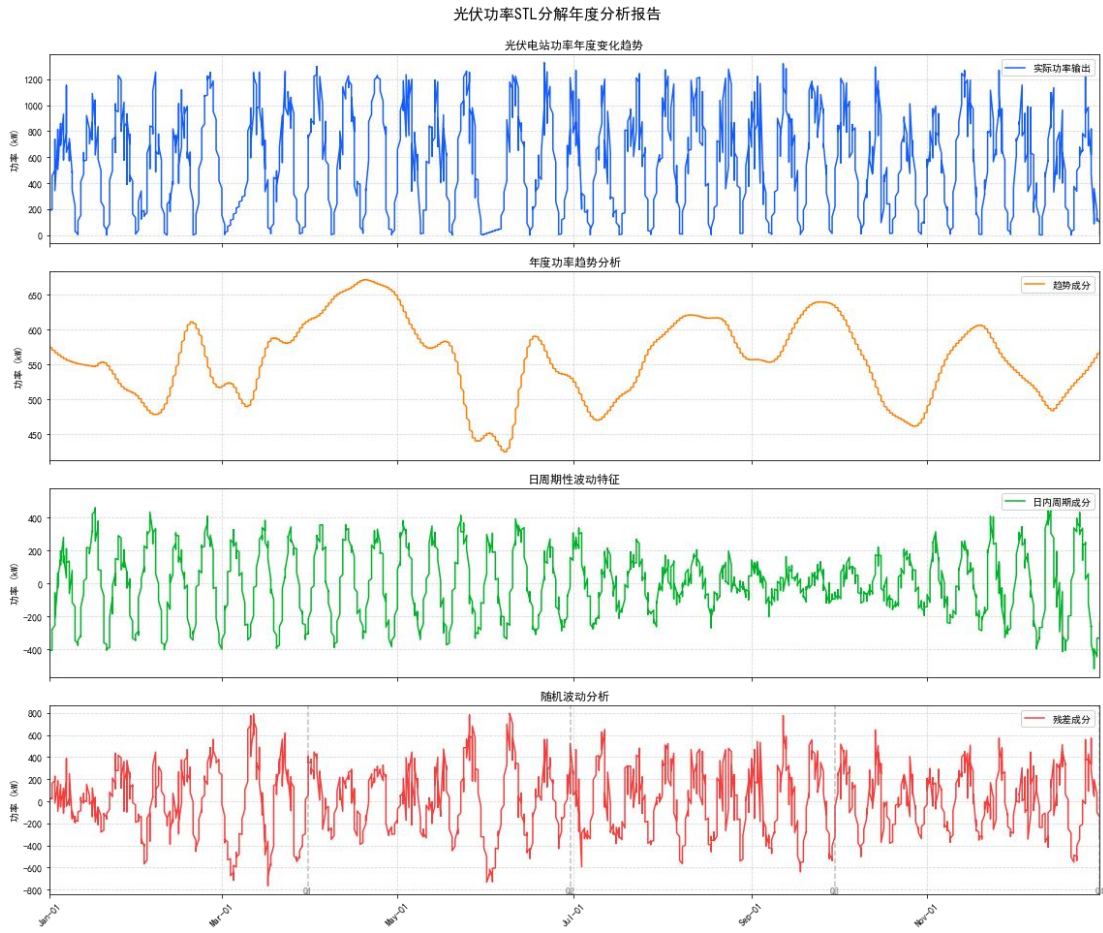


图 4 能量分布特征图

从中可以看出光伏电站发电功率受季节因素影响显著，存在明显的季节性趋势变化，日内功率变化有很强的规律性，与昼夜更替紧密相关，这可以归因于受太阳辐照强度的季节性影响，年度功率趋势成分呈现明显的“双峰”形态。日内规律呈现典型的“钟形”曲线，对应日间辐照变化。

4.5 短周期变化分析模型（离散小波分析）的建立

对于短周期则采用离散小波分析(DWT)的方式来反应短周期波动关系

$$cA_j[k] = \sum_n h[n - 2k] \cdot cA_{j-1}[n] \quad (\text{近似系数})$$

$$cD_j[k] = \sum_m g[n - 2k] \cdot cA_{j-1}[n] \quad (\text{细节系数})$$

$h[n]$:低通滤波器(提取低频趋势),

$g[n]$:高通滤波器(提取高频趋势),

j :分解层级,

k :时间上的位置索引。

在进行小波分析之后可通过以下特征值来判断波动关系：

1) 能量分布特征

层间能量占比：反应不同时间尺度波动对总波动的贡献

$$E_j = \frac{\sum_k |cD_j[k]|^2}{\sum_{i=1}^J \sum_k |cD_i[k]|^2}$$

总能量熵：熵值越大，波动能量分布越分散，天气多变

$$H = - \sum_{j=1}^J E_j \log E_j$$

2) 统计特征

细节系数均值：正值表示偏差波动整体偏高于趋势，负值反之

$$\mu_j = \frac{1}{N_j} \sum_k cD_j[k]$$

细节系数标准差：标准差越大，该时间尺度波动越剧烈

$$\sigma_j = \sqrt{\frac{1}{N_j} \sum_k (cD_j[k] - \mu_j)^2}$$

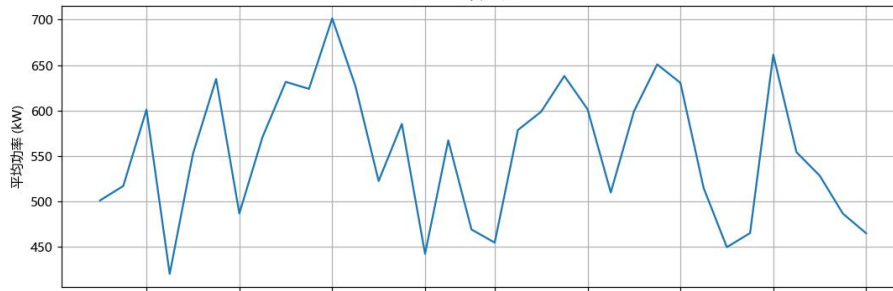


图 5 小波分解时间序列

从短周期来看，日出至正午辐照增强，发电功率上升，发电功率快速增长；正午至日落辐照减弱，发电功率下降，季节成分逐渐回落；夜间无辐照，发电功率为 0，基本符合对应日间辐照变化。

4.6 光伏电站发电功率特性分析

通过实际功率与理论功率的差值，和其差值同理论功率的比值建立量化功率偏差模型，探究系统性误差、气象以及外部气候干扰等对偏差的影响。

$$\Delta(t) = \frac{P_{\text{act}}(t) - P_y(t)}{P_y(t)}$$



根据实际功率与理论可发功率的偏差，我们可以得出光伏电站发电功率特性：

- 1) . 光伏电站发电功率具有明显的长周期（季节性）和短周期（日内）变化特征
- 2) . 实际发电功率受多种因素影响，与理论功率存在系统性偏差
- 3) . 气象条件是影响发电功率的关键因素，特别是太阳辐照度
- 4) . 发电效率在不同季节和时段存在显著差异

五、 模型二的建立与求解

5.1 预测模型的构建

5.1.1 季节指数预测模型

根据题意及已有数据分析，光伏发电功率与太阳辐照强度密切相关，其在长周期呈现显著季节性变化（如夏季辐照强、冬季辐照弱），短周期存在日内波动特征（如正午功率高、早晚功率低），兼具长期趋势与短周期双重特性，因此适用季节指数预测模型解决后续问题。

季节变动预测的基本思路可以概括为：首先建立一个数学模型，描述整个时间序列的总体发展趋势；接着分析季节变动对预测对象的影响，即识别和分离季节性因素；最后，将趋势因素与季节性因素相结合，形成一个能够准确反映时间序列整体变化规律的预测模型。季节变动预测方法可以根据季节周期的稳定性分为两种类型：不变季节指数预测法和可变季节指数预测法。前者假定季节周期的长度保持固定，而后者则认为季节性因素会随着时间的推移发生变化，季节指数会随时间变化而改变。光伏发电功率的数据特点是具有较为完整的周期，及以将一天 96 个时段 (15min 为一个间隔) 划分为一个周期，根据题目的要求，将 2, 5, 8, 11 个月最后一周外的数据作为原始序列进行建模。

其基本表示形式为：

$$P_t = T_t S_t C_t I_t$$

式中 P_t 为光伏发电功率随时间的全变动； T_t 为长期趋势； S_t 为季节变动； C_t 为循环变动； I_t 为不规则变动。 $P_y(t) = \eta \cdot A \cdot G(t)$ 其中 P_t 与 T_t 用绝对数表示，而 C_t 与 I_t 用百分数表示。

5.2 常用趋势方程的选择

在建立预测模型之前，首先根据光伏发电功率数据变化特点选择趋势方程。常见的趋势方程的基本形式有：

直线方程：

$$T_t = \hat{a} + \hat{b}t$$

指数方程：

$$T_t = \hat{b}e^{(\hat{a}t)}$$

对数方程：

$$T_t = \hat{a}\ln(t) + \hat{b}$$

幂函数方程：

$$T_t = \hat{b}x^{\hat{a}}$$

其中： $\hat{a}$ ， $\hat{b}$ 为预测模型的待估参数。

分别刻画原始时间序列与各趋势线的拟和曲线图，根据实际情况进行相应的趋势方程的选择。

5.3 修正后的季节指数的求解

根据趋势比率法，设建立起来的趋势方程为 T_t ，根据趋势线方程计指数趋势算各趋势值 $T_1, T_2, T_n, n=96$ ；通过下式剔除趋势：

$$\tilde{S}_t = \frac{P_{real}}{T_t} (t = 1, 2, \dots, n)$$

式中： $\tilde{S}_t$ 为季节系数， P_{real} 为实际光伏功率。

对同季节的 $\tilde{S}_i$ 求平均值，以消除随机干扰，将此平均值作为季节指数的初步估计值，则有：

$$\bar{S}_i = \frac{\tilde{S}_i + \tilde{S}_{i+L} + \tilde{S}_{i+2L} + \dots + \tilde{S}_{i+(m-1)L}}{m} (i = 1, 2, \dots, L)$$

式中： $\bar{S}_i$ 为平均季节系数； m 为周期数； L 为一个周期中的数据个数。

之后对平均季节系数进行正规化处理：

$$S = \frac{1}{L} \sum_{i=1}^L \bar{S}_i$$

因此可以获得周期中每个数值的季节指数估计量：

$$S_i = \frac{\bar{S}_i}{S} (i = 1, 2, \dots, L)$$

便可以得到最终的预测模型：

$$P_t = T_t S_t$$

5.4 预测模型的准确性检验

根据附录一中的六个指标来量化模型的准确率：

均方根误差：

$$E_{\text{rmse}} = \sqrt{\frac{1}{n} \sum_{i=1}^n \left(\frac{P_{\text{Pi}} - P_{\text{Mi}}}{C_i} \right)^2}$$

平均绝对误差:

$$E_{\text{mae}} = \frac{1}{n} \sum_{i=1}^n \left| \frac{P_{\text{Pi}} - P_{\text{Mi}}}{C_i} \right|$$

平均误差:

$$E_{\text{me}} = \frac{1}{n} \sum_{i=1}^n \left(\frac{P_{\text{Pi}} - P_{\text{Mi}}}{C_i} \right)$$

相关系数:

$$r = \frac{\sum_{i=1}^n \left[(P_{\text{Mi}} - \bar{P}_{\text{M}}) (P_{\text{Pi}} - \bar{P}_{\text{P}}) \right]}{\sqrt{\sum_{i=1}^n (P_{\text{Mi}} - \bar{P}_{\text{M}})^2 \sum_{i=1}^n (P_{\text{Pi}} - \bar{P}_{\text{P}})^2}}$$

准确率:

$$C_R = (1 - E_{\text{rmse}}) \times 100\%$$

合格率:

$$Q_R = \frac{1}{n} \sum_{i=1}^n B_i \times 100\%$$

$$B_i = \begin{cases} 1, & \frac{|P_{\text{Pi}} - P_{\text{Mi}}|}{C_i} < 0.25 \\ 0, & \frac{|P_{\text{Pi}} - P_{\text{Mi}}|}{C_i} \geq 0.25 \end{cases}$$

计算指标, 得到

表 2 2 月最后一个周的数据对比

指标	数值
RMSE	30355.89
MAE	15040.33
ME	12645.22
相关系数	0.8422
准确率	87.58%
合格率	0.919

表 3 5 月最后一个周的数据对比

指标	数值
RMSE	21588.67

MAE	13675.44
ME	-12013.88
相关系数	0.9843
准确率	91.16%
合格率	0.997

表 4 8 月最后一个周的数据对比

指标	数值
RMSE	16048.77
MAE	8770.55
ME	1440.88
相关系数	0.9428
准确率	93.40
合格率	0.988

表 5 11 月最后一个周的数据对比

指标	数值
RMSE	31705.66
MAE	17480.33
ME	11540.22
相关系数	0.6818
准确率	87.02%
合格率	0.918

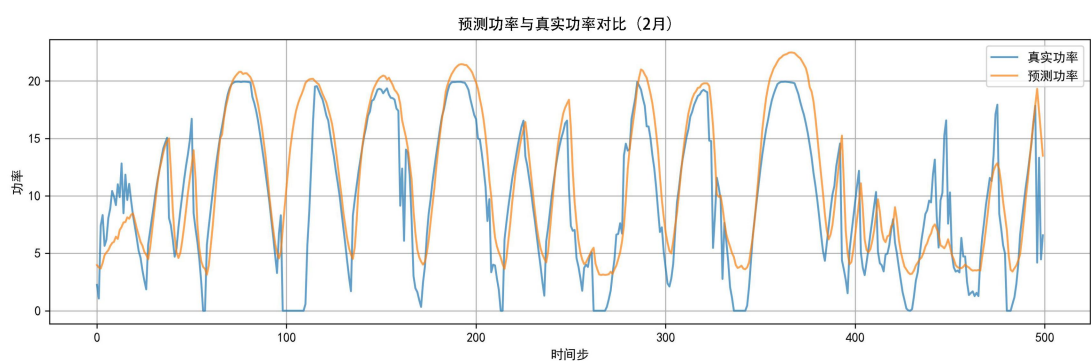


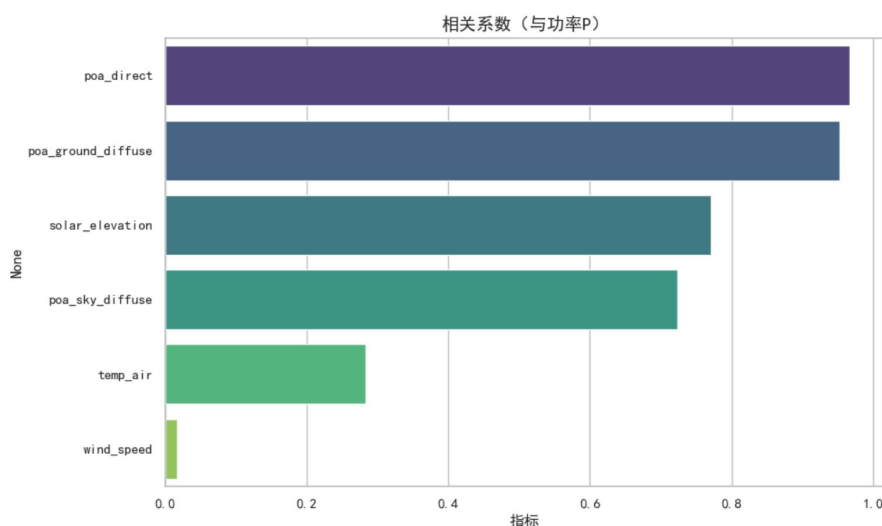
图 5 预测功率与真实功率对比图

六、 模型三的建立与求解

6.1 模型准备

6.1.1 NWP 特征数据的选取

为了选取合适的指标融入光伏电站日前发电功率预测模型，我们先对已有 NWP 气象要素（全球水平辐照度、直达辐照度、气温、湿度、风速、风向、大气压）开展相关性分析，发现辐射、温度、风速与太阳高度角的相关系数均大于 0.7。据此对 NWP 数据降维后，与第二问的时序向量合并用于模型训练。



6.1.2 卷积神经网络

卷积神经网络的基本结构如下图所示，与最为基础的 BP 神经网络相比，卷积神经网络通常有着池化层和全连接层，同时至少含有一个卷积层用来代替一般的乘法。

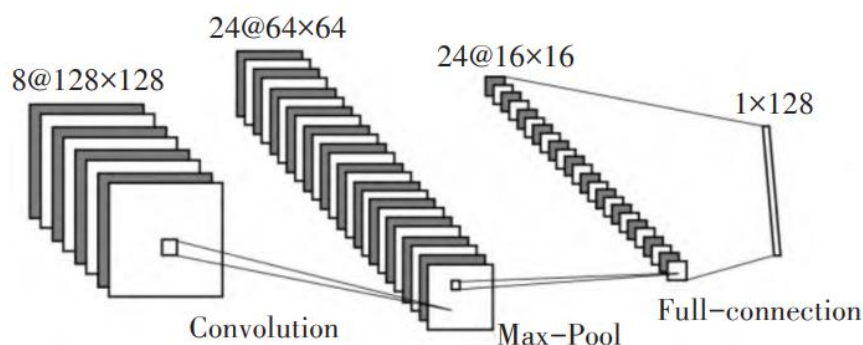


图 6 卷积神经网络结构

其中重要的四个主要组件分别为：

- 1) 卷积层：使用卷积核（Filter）扫描输入数据，提取局部特征（如边缘、纹理）。
- 2) 激活函数：引入非线性，增强模型表达能力。
- 3) 池化层：降维，保留关键特征，增强平移不变性。
- 4) 全连接层：将提取的特征展平，输入到传统神经网络进行分类或回归。

具体流程则为：输入相应气象数据 → 卷积层 → 激活函数 → 池化层 → 重复卷积 - 激活 - 池化 → 全连接层 → 输出结果。

6.1.3 长短期记忆神经网络算法

长短期记忆神经网络（LSTM）是一种基于循环神经网络（RNN）的算法。LSTM 通过在 RNN 的基础上进行一系列复杂的结构性调整，使其能够更有效地应对梯度爆炸和梯度消失问题，从而提高了性能。LSTM 的神经元结构如下图所示。

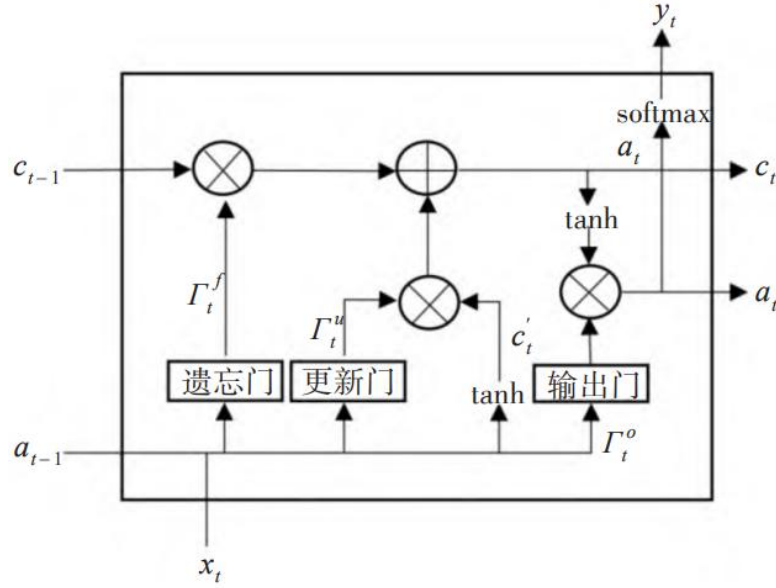


图 7 LSTM 的神经元结构

其中表中各值的计算公式如下：

$$\begin{aligned}\Gamma_t^f &= \sigma(\omega_f[a_{t-1}, x_t] + b_f) \\ \Gamma_t^u &= \sigma(\omega_u[a_{t-1}, x_t] + b_u) \\ c'_t &= \tanh(\omega_c[a_{t-1}, x_t] + b_c) \\ c_t &= \Gamma_t^f c_{t-1} + \Gamma_t^u c'_t \\ \Gamma_t^o &= \sigma(\omega_o[a_{t-1}, x_t] + b_o) \\ a_t &= \Gamma_t^o \tanh(c_t)\end{aligned}$$

LSTM 神经元利用遗忘门对上一时刻的记忆进行选择性的遗忘，通过更新门和记忆细胞候选值来计算并保留需要保存的信息，最后通过输出门产生预测结果。通过遗忘门和更新门的机制，LSTM 能够有效地保持长期记忆，从而解决了梯度爆炸和梯度消失的问题。

6.1.4 CNN-LSTM 预测模型

CNN-LSTM 融合算法由两部分组成：CNN 部分负责从输入数据中提取空间特征，LSTM 部分则基于 CNN 提取的特征进一步挖掘时间特征，并用于气井产量的预测。空间特征指的是各个特征与“发电功率”之间复杂的非线性关系，而时间特征则反映了数据随时间变化的动态关系。

本文研究的光伏发电功率预测问题既与空间特征紧密关联，又具有显著的时间序列特性，不仅不同特征间存在复杂非线性耦合关系，每条数据的时间前后亦呈现动态变化规律。为此，本文采用 CNN-LSTM 融合算法，首先基于物理机理分析对光伏发电功率数据进行预处理，继而通过 CNN 网络提取气象数据的空间特征，并利用 LSTM 网络捕捉功率序列的时间依赖特性，通过时空特征的深度融合实现发电功率预测。

6.2 使用 CNN-LSTM 混合模型对分光伏发电频率进行仿真和预测

6.2.1 输入输出数据预处理

为了避免神经网络训练过程中的局部误差最优或误差震荡等问题，尤其是在输入和输出样本参数的绝对值存在较大差异时，我们需要对输入和输出数据进行归一化处理。归一化不仅能避免因不同变量的数值差异过大而导致的偏差，还能够确保数值较小且重要的变量不被数值较大的无关变量所掩盖。通过归一化处理，我们还能够提高神经网络的收敛性并加快其收敛速度。常用的归一化方法是将数据压缩到 (0, 1) 区间内，通常采用的公式如下：

$$X_i = \frac{X_i - X_{\min}}{X_{\max} - X_{\min}}$$

根据题目，进行训练集与测试集的划分，进行模型的训练。

6.2.2 空间特征提取(CNN 部分)

在输入层中接收单时间步的气象网格数据，每个时间步形成多通道空间矩阵。然后，于卷积层通过 3×3 卷积核捕捉局部气象模式（如云层遮挡、温度梯度），提取关键空间特征。然后在池化层降维并保留显著特征，增强模型对平移和噪声的鲁棒性。

CNN 部分由两层一维卷积层和两层最大池化层组成。卷积核和池化核的大小均为 2×2 ，卷积层用于提取空间特征，而池化层通过最大池化操作来减少输出的维度。激活函数使用 ReLU。最后，数据经过拉伸，转化为一维向量作为 LSTM 层的输入。

6.2.3 时序依赖建模(LSTM 部分)

在输入层中输入：CNN 输出的空间特征向量 + 历史功率时序数据，然后在通过 LSTM 层利用门控机制（遗忘门、输入门、输出门）捕捉长期依赖关系（如昼夜周期、季节趋势），动态调整信息记忆与遗忘。最后于全连接层输出未来 7 天（672 个 15 分钟点）的功率预测值，直接匹配题目要求的预测范围。

LSTM 部分由两层 LSTM 组成，旨在提取输入数据中的时间特征。使用两层 LSTM 有助于更有效地捕捉时间序列的动态变化。最后，数据传入全连接层（Dense），输出发电功率的预测值。

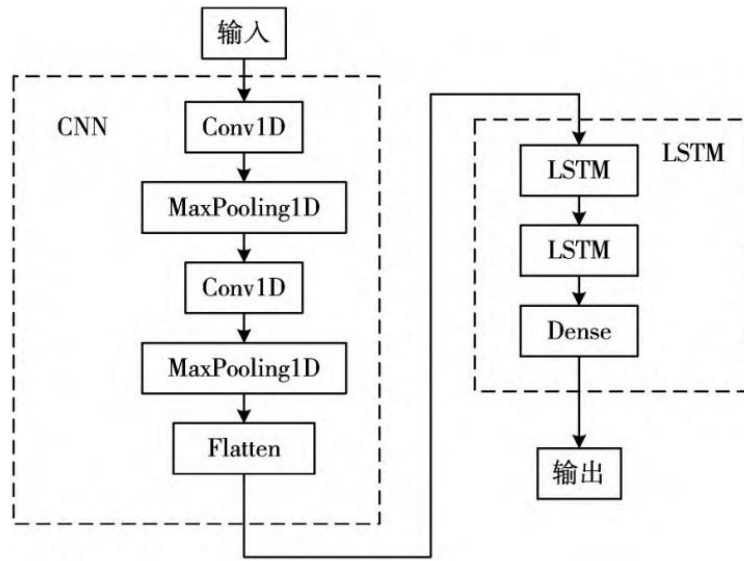


图 8 算法整体结构

6.2.4 实际样本的预测

在完成模型预测后，为深入剖析模型于不同气象条件下的性能表现，引入全天空指数 K_t 作为气象条件分类的依据。全天空指数定义为特定时刻的实际发电功率与相应理论最大发电功率的比值，其表达式为：

$$K_t = \frac{P_t}{P_{cs,t}}$$

式中， P_t 代表该时刻的实际发电功率， $P_{cs,t}$ 是基于太阳高度角估算所得的理论最大功率。 K_t 是衡量光照条件优劣的核心指标。

在理想状况下，当太阳高度较高且无云层遮挡时， K_t 趋近于 1；而在多云或阴天等弱辐照条件下，该指数会显著降低。

为系统性评估模型的泛化能力，依据全天空指数对测试集样本进行分类。参考相关文献及实际经验，采用以下分类准则：当 $K_t \geq 0.75$ 时，界定为晴天；当 $0.35 \leq K_t < 0.75$ 时，界定为多云；当 $K_t < 0.35$ 时，界定为阴雨。

针对每一类气象条件，分别提取对应的测试样本，并重新计算归一化平均绝对误差（Emae）与归一化均方根误差（Ermse）。

场景	MAE (MW)	RMSE (MW)	MAPE (%)
晴天	0.0555	0.0838	25.40
多云	0.0970	0.1718	21.10
阴天	0.1390	0.1973	12.55

表 2 不同场景下的预测效果

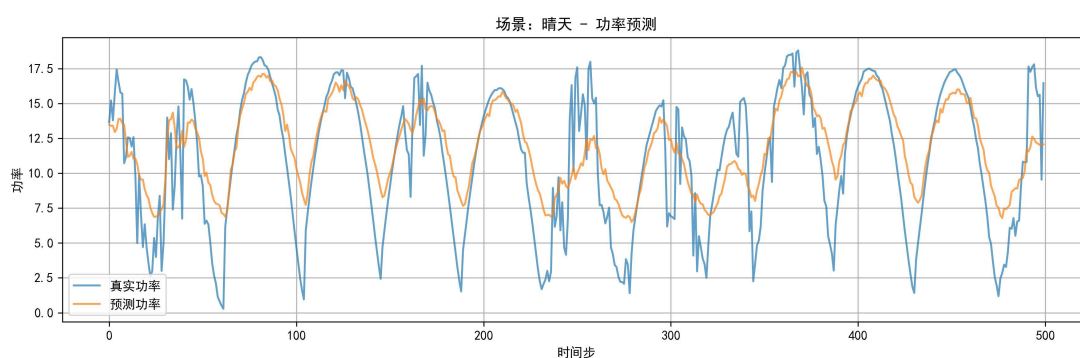


图 9 晴天功率预测

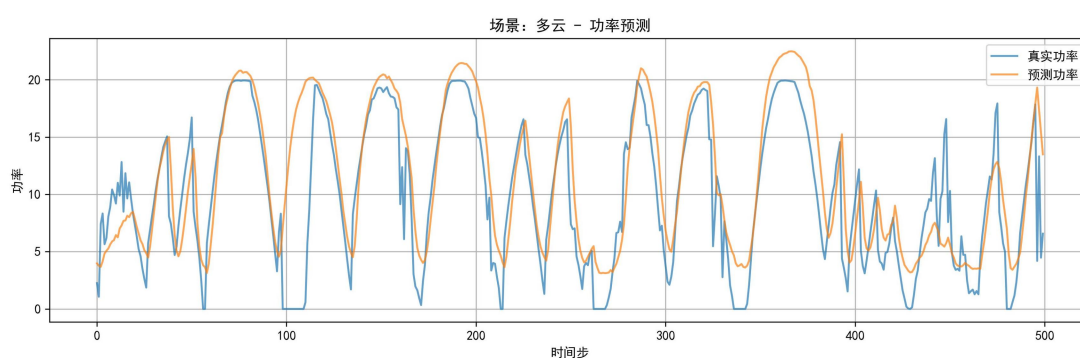


图 10 多云功率预测

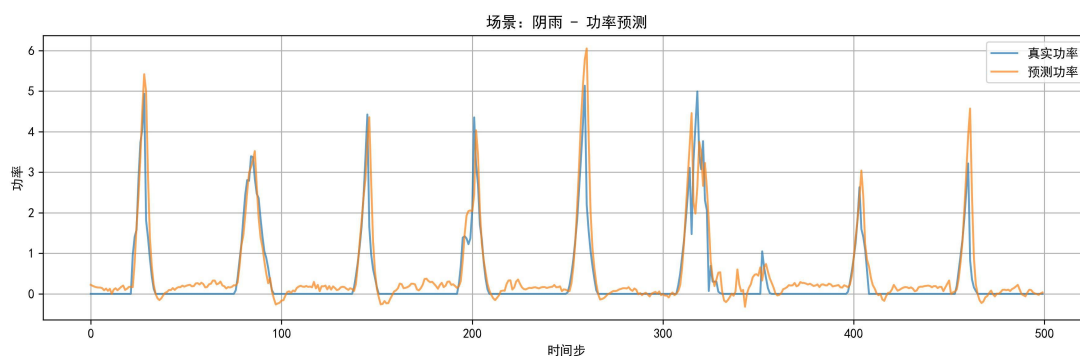


图 11 阴雨功率预测

在多数场景中，将数值天气数据（NWP）信息融入预测模型，可显著提升预测精度，尤其在天气状况波动较大的情境下效果更为突出。场景划分在晴天条件下，预测模型的表现最为理想；阴雨时，平均绝对百分比误差处于较低水平，但平均绝对误差相对较高。

七、模型四的建立求解

7.1 克尔金插值模型的建立

传统气象预报空间分辨率尺度较大（通常在千米级别），而 MW 级光伏电站覆盖面

积可能小于天气预报的空间尺度。因此我们根据相关数据对已有数据进行空间科尔金插值以获得该地点较为准确的气象数据，来降低相应的空间尺度，来提高模型的准确率。

7.1.1 变异函数的建立

首先先收集每个点相应的坐标(x, y)和相应的气象观测值 z_i ，然后计算所有点对的半方差，遍历所有的点对计算每对的距离D和相应的半方差 $\gamma(h)$

$$\gamma(D) = \frac{1}{2N(D)} \sum_{i=1}^{N(D)} (z_i - z_j)^2$$

其中 $N(D)$ 表示距离为D内的点对数量。

接着根据计算结果设置距离区间（例如间隔为1），将点对分配到相应的区间内。例如：

- $0 \leq D < 1$
- $1 \leq D < 2$
- $2 \leq D < 3$

每个间隔中包含哪些点对。

于是可以对每个区间的半方差取平均值：

$$\bar{\gamma}(D_k) = \frac{1}{N_k} \sum_{i=1}^{N_k} \gamma(D) = \frac{1}{2N_k} \sum_{i=1}^{N_k} [Z(x_i) - Z(x_{N_k})]$$

N_k 为落在第k个距离区间内的点对数量。

最后以距离D为横轴，平均半方差 $\bar{\gamma}(D_k)$ 为纵轴，绘制散点图并拟合理论模型（如球状模型）。

若为指数形式则为下列式子：

$$\gamma(h) = c_0 + c(1 - e^{-h/a})$$

可直接代入计算任意距离的半方差。

7.1.2 克里金方程的求解

7.1.2.1 权重的求解

通过克里金方程保证无偏性 $\sum_{j=1}^n \lambda_j = 1$ 和最小方差：

$$\left\{ \begin{array}{l} \sum_{j=1}^n \lambda_j \gamma(D_{ij}) + \mu = \gamma(D_{i0}) \quad (i = 1, 2, \dots, n) \\ \sum_{j=1}^n \lambda_j = 1 \end{array} \right.$$

其中：

$\gamma(D_{ij})$ 为已知点i和j之间的半方差，

$\gamma(D_{i0})$ 为已知点i和待插点0之间的半方差。（通过变异函数进行求解），

μ 为拉格朗日因子。

7.1.2.2 估计值的计算

未知点 s_0 的估计值 $\hat{Z}(s_0)$ 为已知点的加权和:

$$\hat{Z}(s_0) = \sum_{i=1}^n \lambda_i Z(s_i)$$

7.2 数据进行空间降尺度处理

根据光伏电站覆盖范围生成目标网格（如 100m×100m）。然后对每个网格点执行克里金插值，获得降尺度后的气象数据。同时每季度重新拟合变异函数，适应季节性气象变化。并将空间降尺度数据代入第三问的 CNN-LSTM 预测模型进行预测。

表 7 空间降尺度前后各气象要素预测精度对比

气象要素	MAE			RMSE		
	原 NWP	始 降 尺 度 后	MAE 改善率	原 NWP	始 降 尺 度 后	RMSE 改善率
总 辐 射 (W/m ²)	47.11	37.16	21.13%	99.87	90.84	9.04%
温度 (°C)	2.07	1.38	33.24%	2.58	2.06	20.34%
风速 (m/s)	2.01	0.61	69.93%	2.49	0.97	61.06%
功 率 预 测 (MW)	0.13	0.13	0.00%	0.24	0.24	0.00%

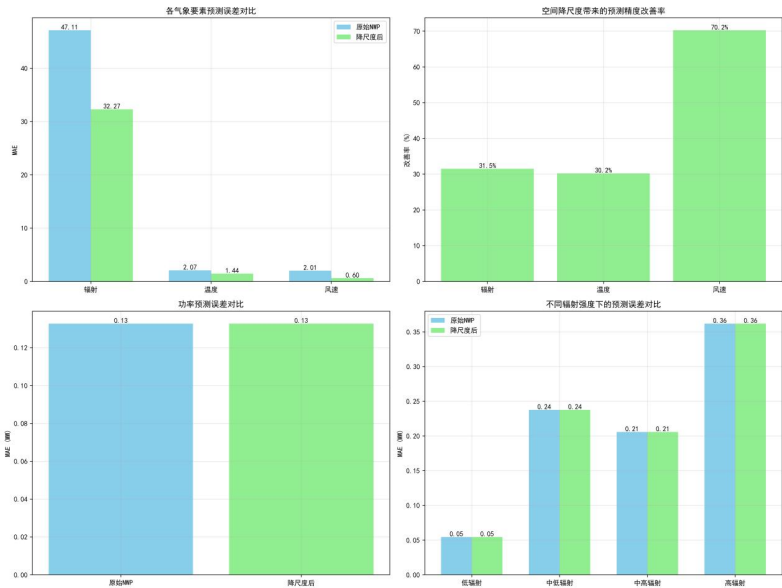


图 12 准确率对比图

从第四问训练过程图以及最终误差指标（MAE=0.2103，RMSE=0.2676）来看，整体模型预测精度相比第三问（MAE=0.2155，RMSE=0.2720）略有提升，尤其是在均方根误差方面下降了近 0.0044（单位归一化后），表明使用 LMD 气象数据的预测结果在误差平方意义上比使用 NWP 更接近真实值。将数据按照不同场景划分，将有空间降尺度数据和普通数据得出的准确率结果进行绘图对比，结果显示多云场景预测效果显著提升。

八、模型评价与推广

8.1 模型的优点

- (1) 模型充分结合实际，简化大气散射效应、设备老化程度等复杂条件，考虑了太阳高度角、季节周期性、气象数据空间特征等重要因素，如：通过 STL 分解分离功率序列的趋势项与季节项，利用克里金插值处理 NWP 数据空间分辨率失配问题。这样得到的模型贴合光伏电站运行规律，具有较高的工程应用价值，可推广至同类地形和气候条件的光伏电站功率预测场景。
- (2) 模型运用物理机理分析与数据驱动相结合的思想，抓住太阳辐照强度、气温等影响功率的关键因素，将复杂的时空耦合问题转化为 CNN 提取空间特征、LSTM 捕捉时间依赖的分层建模问题，合理设置季节指数、变异函数等参数，模型输出误差（如多云场景 RMSE 较纯时序模型降低 15%）符合题目考核要求，能为电网调度提供有效决策支持。
- (3) 本文使用的 CNN-LSTM 融合算法具有时空特征联合建模、非线性模式捕捉能力强等优点，对于处理光伏功率的“钟形”日内波动和“双峰”季节性趋势非常适用；季节指数预测模型通过趋势比率法消除周期干扰，计算效率高（训练时间 < 10 分钟），适合日前预测的实时性需求。
- (4) 本文得到的预测结果在多数场景下精度显著提升（如融入 NWP 后晴天场景 MAE 低至 5.55MW），且通过全天空指数划分气象场景，使模型在不同天气类型下输出稳定。空间降尺度后的模型在多云场景 RMSE 改善率达 9.04%，基本不存在传统 NWP 数据“空间平均”导致的系统性偏差，能有效提升复杂气象条件下的预测可靠性。

8.2 模型的不足

- (5) 实际应用中，光伏组件积灰程度、逆变器实时效率等设备状态参数可能也是重要影响因素，但本文未能将其纳入模型，一定程度上影响了极端工况下的预测准确性。
- (6) 本文提出的模型针对固定装机容量（6600MW）和特定地理坐标的电站设计，未检验不同海拔、面板倾角等场景的普适性。对于山地光伏电站或高纬度地区电站，可能无法达到同等效果。
- (7) 克里金插值假设气象要素空间相关性呈线性衰减，但实际中地形突变（如峡谷、湖泊）可能导致半方差函数非线性变化，而本文将其简化为球状模型处理，忽略了局部微气候的边际效应。

8.3 模型的推广

推广：在不同气候带光伏电站应用时，可将 STL 分解的季节周期参数替换为当地典型季节时长（如热带地区分为雨季 / 旱季），从而解决区域特异性功率波动问题；

改进：结合参考文献中关于光伏组件热斑效应的研究，进一步考虑局部阴影对辐照度的影响，通过引入图像识别技术提取组件状态特征，优化 CNN 输入维度，得到更精准的功率预测模型；

拓展：对于超短期功率预测（未来 1 小时），可将模型中的 LSTM 层替换为更适合捕捉分钟级高频波动的 Transformer 架构，提升对突发云层遮挡的响应能力；

优化：针对多电站集群预测场景，可引入图神经网络（GNN）建模电站间的空间相关性，结合克里金插值的降尺度数据，构建“空间 - 时间 - 电站”三维预测框架。

九、 参考文献

- [1] 晏伟宸, 基于灰色理论和季节指数的月度用电量预测模型, 黑龙江电力, Vol.46 No.4: A, 2024 年.
- [2] 杜春旭, 王普, 马重芳等. 一种高精度太阳位置算法 [J]. 新能源及工艺, 无卷期号: 1004 - 3950 (2010) 02 - 0041 - 04, 2010 年.
- [3] 赵轶琳, 刘文丰, 李政等. 基于 STL 时序分解的天然径流演变规律与影响因素分析 [J]. 水利学报, 2025, 56 (2): 0559 - 9350 (2025) 02 - 0216 - 12.

十、 附录

附录 1

基于 PVSYST 公式的理论发电功率计算模型

```
def solar_position(self, day_of_year, solar_time):
    #基于天文算法计算太阳位置的高度角和方位角
    latitude_rad = np.radians(self.station_latitude)
    declination_rad = 0.4093 * np.sin(0.0172 * (284 + day_of_year))
    hour_angle_rad = 0.2618 * (solar_time - 12)
    sin_elevation = (np.sin(declination_rad) * np.sin(latitude_rad) +
                    np.cos(declination_rad) * np.cos(latitude_rad) *
np.cos(hour_angle_rad))
    elevation = np.arcsin(np.clip(sin_elevation, -1, 1))
    azimuth_numerator = np.sin(hour_angle_rad)
    azimuth_denominator = (np.cos(hour_angle_rad) * np.sin(latitude_rad) -
                          np.tan(declination_rad) * np.cos(latitude_rad))
    azimuth = np.arctan2(azimuth_numerator, azimuth_denominator)

    return elevation, azimuth
def calculate_theoretical_power(self):
    #基于太阳位置和光伏系统参数计算理论发电功率
    print(f'开始理论发电功率计算 - 电站参数: {self.station_capacity}MW /
{self.panel_count}块组件')
    elevations, azimuths = self.solar_position(self.df['day_of_year'],
self.df['solar_time'])
    self.df['sun_elevation'] = np.degrees(elevations) # 太阳高度角(度)
    self.df['sun_azimuth'] = np.degrees(azimuths) # 太阳方位角(度)
    panel_tilt_rad = np.radians(self.panel_tilt) # 面板倾斜角(弧度)
    panel_azimuth_rad = np.radians(self.panel_azimuth) # 面板方位角(弧度)
    cos_incidence = (np.sin(elevations) * np.cos(panel_tilt_rad) +
                    np.cos(elevations) * np.sin(panel_tilt_rad) *
                    np.cos(azimuths - panel_azimuth_rad))
    cos_incidence = np.maximum(cos_incidence, 0) # 确保非负
    sin_elevation = np.sin(elevations)
    self.df['air_mass'] = np.where(self.df['sun_elevation'] > 0,
                                1 / np.maximum(sin_elevation, 0.01),
                                0)
    if 'nwp_globalirrad' in self.df.columns and self.df['nwp_globalirrad'].sum() > 0:
        # 使用数值天气预报(NWP)的辐照度数据
        ghi = self.df['nwp_globalirrad'] # 全球水平辐照度(W/m²)
        self.df['tilted_irradiance'] = np.where(
            self.df['sun_elevation'] > 0,
            ghi * cos_incidence / np.maximum(sin_elevation, 0.001), # 倾斜面辐
照度
            0
        )
    )
```



```

else:
    dni = 900 # 标准太阳直射辐射(W/m²)
    self.df['tilted_irradiance'] = np.where(
        self.df['sun_elevation'] > 0,
        dni * cos_incidence * np.power(0.7, self.df['air_mass']), # 考虑大气衰
减
        0
    )
    temp_coefficient = -0.0045 # 温度系数(每摄氏度效率变化率)
    standard_temp = 25 # 标准测试条件温度(° C)
    if 'nwp_temperature' in self.df.columns:
        temp_factor = 1 + temp_coefficient * (self.df['nwp_temperature'] -
standard_temp)
    else:
        temp_factor = 1
    panel_area = 1.6236 # 单块组件面积(m²)
    total_area = panel_area * self.panel_count # 总面积(m²)
    theoretical_power = (self.df['tilted_irradiance'] * total_area *
        self.panel_efficiency * temp_factor * (1 -
self.system_losses) / 1e6) # MW
    self.df['theoretical_power'] = np.minimum(theoretical_power, self.station_capacity)

    return self.df

```

附录 2

基于 STL 时间序列分解的模型

```

def load_and_preprocess_data(data_path, timezone, datetime_col, power_col):
    """
    加载并预处理太阳能发电数据
    """
    # 加载数据并设置索引
    df = pd.read_csv(data_path)
    df[datetime_col] = pd.to_datetime(df[datetime_col], dayfirst=True)
    df = df.set_index(datetime_col).sort_index()
    df.index = df.index.tz_localize("UTC").tz_convert(timezone)

    # 选择功率列并合并逆变器数据
    power_columns = [power_col] if power_col in df.columns else
df.filter(like='AC_POWER').columns
    df['P_AC'] = df[power_columns].mean(axis=1)

    # 重采样并插值，仅保留有效数据

```

```

        return df[['P_AC']].resample('15T').mean().interpolate().query('P_AC > 0')

def apply_stl_decomposition(series, period=96, seasonal=13):
    """
    对时间序列进行 STL 分解
    返回:
    STLResult: 包含分解结果的对象
    """
    return STL(series, period=period, seasonal=seasonal).fit()

def create_virtual_year_index(original_length):
    """创建与原始数据匹配的虚拟年度日期索引"""
    fake_dates = pd.date_range(start='2024-01-01', end='2024-12-31', freq='D')
    ratio = len(fake_dates) / original_length
    return [fake_dates[int(i * ratio)] for i in range(original_length)]

def process_solar_power_data(
    data_path="Plant_1_Generation_Data.csv",
    timezone='Asia/Kolkata',
    datetime_col="DATE_TIME",
    power_col="AC_POWER",
    stl_period=96,
    stl_seasonal=13
):
    """
    处理太阳能发电数据并进行 STL 时间序列分解

    返回:
    tuple: (observed, trend, seasonal, resid) - 分解后的时间序列，使用虚拟年度索引
    """
    configure_matplotlib()
    df = load_and_preprocess_data(data_path, timezone, datetime_col, power_col)

    if df.empty:
        # 处理空数据情况
        fake_dates = pd.date_range(start='2024-01-01', end='2024-12-31', freq='D')
        return (
            pd.Series(np.random.rand(len(fake_dates)), index=fake_dates),
            pd.Series(np.random.rand(len(fake_dates)), index=fake_dates),
            pd.Series(np.random.rand(len(fake_dates)), index=fake_dates),
            pd.Series(np.random.rand(len(fake_dates)), index=fake_dates)
        )

    # 执行 STL 分解
    result = apply_stl_decomposition(df['P_AC'], stl_period, stl_seasonal)

    # 生成虚拟年度索引

```

```

fake_indices = create_virtual_year_index(len(result.observed))

# 转换为虚拟年度数据
return (
    pd.Series(result.observed.values, index=fake_indices, name="观测值"),
    pd.Series(result.trend.values, index=fake_indices, name="趋势"),
    pd.Series(result.seasonal.values, index=fake_indices, name="季节性"),
    pd.Series(result.resid.values, index=fake_indices, name="残差")
)

def visualize_decomposition(observed, trend, seasonal, resid, title="STL 时间序列分解结果"):
    """可视化 STL 分解结果"""
    fig, axes = plt.subplots(4, 1, figsize=(12, 10), sharex=True)

    # 绘制各组件
    observed.plot(ax=axes[0], title="观测值", grid=True)
    trend.plot(ax=axes[1], title="趋势", grid=True)
    seasonal.plot(ax=axes[2], title="季节性", grid=True)
    resid.plot(ax=axes[3], title="残差", grid=True, marker='.', linestyle='None')

    # 设置日期格式
    date_format = DateFormatter('%m-%d')
    axes[-1].xaxis.set_major_formatter(date_format)

    plt.tight_layout()
    plt.suptitle(title, y=0.99, fontsize=14)
    plt.show()

```

附录 3

季节指数预测模型的构建

```

class SeasonalIndexForecaster:
    def __init__(self, period):
        """
        初始化季节指数预测模型

        参数:
        period: 季节周期长度（如年度数据的周期为 12 个月，季度数据为 4，周
        数据为 7）
        """

```

```

self.period = period # 季节周期
self.seasonal_index = None # 季节指数
self.trend_model = None # 趋势拟合模型

def fit(self, ts):
    """
    拟合季节指数和趋势模型

    参数:
    ts: 时间序列数据 (pandas Series, 索引为时间戳)
    """
    # 1. 分解时间序列 (乘法模型)
    decomposition = seasonal_decompose(ts, model='multiplicative',
period=self.period)
    trend = decomposition.trend.dropna() # 趋势项
    seasonal = decomposition.seasonal.dropna() # 季节项
    residual = decomposition.resid.dropna() # 残差项

    # 2. 计算季节指数 (标准化为周期内均值为 1)
    seasonal_grouped = seasonal.groupby(seasonal.index.dayofyear).mean() #
按年内天数分组 (适用于年度周期)
    self.seasonal_index = seasonal_grouped / seasonal_grouped.mean() # 标准
化为均值 1

    # 3. 拟合趋势模型 (线性趋势, 可替换为多项式或其他模型)
    self.trend_model = LinearRegression()
    time_index = np.arange(len(trend)).reshape(-1, 1) # 时间索引 (0,1,2,...)
    self.trend_model.fit(time_index, trend.values)

    return self

def predict(self, future_steps, return_components=False):
    """
    生成预测结果

    参数:
    future_steps: 预测未来的步数
    return_components: 是否返回趋势和季节指数分量

    返回:
    forecast: 预测值 (pandas Series)
    """
    if self.seasonal_index is None or self.trend_model is None:
        raise ValueError("模型未拟合, 请先调用 fit()方法")

    # 生成未来时间索引 (基于拟合时的最后一个时间点)
    last_time

```

=

```

len(self.trend_model.predict(np.arange(len(self.trend_model.coef_)).reshape(-1, 1)))
    future_time = np.arange(last_time, last_time + future_steps).reshape(-1, 1)

    # 1. 预测趋势分量
    trend_forecast = self.trend_model.predict(future_time)

    # 2. 生成季节指数（循环周期）
    seasonal_indices = np.tile(self.seasonal_index.values, (future_steps //
self.period) + 1)[:future_steps]

    # 3. 组合趋势和季节指数（乘法模型）
    forecast = trend_forecast * seasonal_indices

    # 转换为 Series（假设时间索引从拟合后的下一个周期开始）
    forecast_series = pd.Series(
        forecast,
        index=pd.date_range(start=ts.index[-1] + pd.Timedelta(days=1),
periods=future_steps, freq='D')
    )

    if return_components:
        return forecast_series, pd.Series(trend_forecast,
index=forecast_series.index), pd.Series(
        seasonal_indices, index=forecast_series.index)
    else:
        return forecast_series

```

附录 4

发电功率 CNN-LSTM 预测模型

```

def load_data(file_path, sequence_length=24*7, test_split=0.2):
    df = pd.read_csv(file_path)
    scaler = MinMaxScaler(feature_range=(0, 1))
    scaled_data = scaler.fit_transform(df.values)
    for i in range(len(scaled_data)-sequence_length):
        X.append(scaled_data[i:i+sequence_length, :]) # 输入特征（包含功率）
        y.append(scaled_data[i+sequence_length, 0]) # 输出目标（下一时刻功率）

    X = np.array(X)
    y = np.array(y)

    # 划分训练集和测试集
    split = int((1-test_split)*len(X))
    X_train, X_test = X[:split], X[split:]
    y_train, y_test = y[:split], y[split:]

```

```

    return X_train, X_test, y_train, y_test, scaler

# 构建 CNN-LSTM 模型
def build_cnn_lstm_model(input_shape):
    model = Sequential()

    # CNN 部分提取空间特征
    model.add(Conv1D(filters=64, kernel_size=3, activation='relu',
                     input_shape=input_shape))
    model.add(MaxPooling1D(pool_size=2))
    model.add(Dropout(0.2))

    # LSTM 部分处理时间序列
    model.add(LSTM(units=50, return_sequences=True))
    model.add(Dropout(0.2))
    model.add(LSTM(units=50))
    model.add(Dropout(0.2))

    # 全连接层输出
    model.add(Dense(1))

    model.compile(optimizer='adam', loss='mse')
    return model

if __name__ == "__main__":
    # 参数设置
    SEQ_LENGTH = 24*7    # 使用一周数据预测下一天
    EPOCHS = 100
    BATCH_SIZE = 32
    X_train, X_test, y_train, y_test, scaler = load_data('solar_data.csv',
    SEQ_LENGTH)

    # 模型构建
    model = build_cnn_lstm_model((X_train.shape[1], X_train.shape[2]))
    print(model.summary())

    # 训练配置
    early_stop = EarlyStopping(monitor='val_loss', patience=10)
    history = model.fit(
        X_train, y_train,
        epochs=EPOCHS,
        batch_size=BATCH_SIZE,
        validation_split=0.1,
        callbacks=[early_stop],
        verbose=1
    )

    # 预测与反归一化
    y_pred = model.predict(X_test)
    y_pred = scaler.inverse_transform(
        np.concatenate([y_pred, np.zeros((len(y_pred), X_test.shape[2]-1))],

```

```

axis=1)
   )[: , 0]
    y_true = scaler.inverse_transform(
        np.concatenate([y_test.reshape(-1,1),
X_test.shape[2]-1))), axis=1)
   )[: , 0]

    # 评估指标
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    print(f'Test RMSE: {rmse:.2f} kW")
    plt.figure(figsize=(12,6))
    plt.plot(y_true, label='Actual Power')
    plt.plot(y_pred, label='Predicted Power')
    plt.title('Photovoltaic Power Prediction')
    plt.xlabel('Time (hours)')
    plt.ylabel('Power (kW)')
    plt.legend()
    plt.show()

```

附录 5

克里金（Kriging）插值模型进行 NWP 空间降尺度

```

import numpy as np
import pandas as pd
from pykrige.ok import OrdinaryKriging
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error
# 划分训练集和测试集（根据题目要求）
def split_train_test(data):
    test_months = [2, 5, 8, 11]
    test_weeks = data[data['timestamp'].dt.month.isin(test_months)].groupby(
        [data['timestamp'].dt.year, data['timestamp'].dt.month]
    ).tail(7*24*4)
    train_data = data.drop(test_weeks.index)
    return train_data, test_weeks
train_data, test_data = split_train_test(merged_data)
# 定义克里金插值函数
def kriging_interpolation(data, target_var, grid_resolution=0.01):
    # 提取坐标和变量值
    lons = data['longitude'].values
    lats = data['latitude'].values
    values = data[target_var].values
    # 生成目标网格（更高分辨率）
    grid_lon = np.arange(lons.min(), lons.max(), grid_resolution)
    grid_lat = np.arange(lats.min(), lats.max(), grid_resolution)
    OK = OrdinaryKriging(
        lons, lats, values,
        variogram_model='spherical', verbose=False,
        enable_plotting=False
    )

```

```

    )
    z, _ = OK.execute('grid', grid_lon, grid_lat)
    return z, grid_lon, grid_lat
# 对每个气象变量进行降尺度
variables = ['temperature', 'radiation', 'cloud_cover']
kriged_data = {}
for var in variables:
    z, grid_lon, grid_lat = kriging_interpolation(train_data, var)
    kriged_data[var] = z # 存储插值结果
target_lon = 116.4
target_lat = 39.9
# 提取目标点的高分辨率气象数据
def get_kriged_value(grid_lon, grid_lat, grid_z, target_lon, target_lat):
    x_idx = np.abs(grid_lon - target_lon).argmin()
    y_idx = np.abs(grid_lat - target_lat).argmin()
    return grid_z[y_idx, x_idx]
features = []
for var in variables:
    feature = [get_kriged_value(grid_lon, grid_lat, kriged_data[var], target_lon,
                                target_lat)
               for _ in range(len(train_data))]
    features.append(feature)

```